

CS336 Assignment 1 Writeup

Author: Mangesh Raut

Course: Stanford CS336 — Language Modeling from Scratch (self-study)

1. Overview

This submission implements a GPT-style language model stack from scratch: byte-level BPE tokenization, transformer primitives, AdamW + cosine learning-rate schedule, checkpointing, and end-to-end training on TinyStories and OpenWebText.

Code entry points:

- `cs336_basics/` — library (tokenizer, model primitives, `train_model.py`, optimizer, data)
- `train.py` — training loop with memmap, wandb, generation
- `scripts/` — BPE training, tokenization, experiments, `run_all.sh`

Artifacts: `artifacts/` (tokenizers, token bins, experiment outputs)

2. BPE Tokenizer (§2.5–2.7)

2.5 Training

We train two tokenizers:

Tokenizer	Corpus	Vocab size	Special token
TinyStories	TinyStoriesV2-GPT4-train.txt	10,000	<code><\ endoftext\ ></code>
OpenWebText	owt_train_2gb.txt (2GB Mac subset)	32,000	<code><\ endoftext\ ></code>

Parallel pretokenization: The corpus is split on `<|endoftext|>` boundaries using `find_chunk_boundaries`, then each chunk is pretokenized in parallel workers. Word-frequency counters are merged before the serial merge loop.

Special-token handling: Text is split on special tokens before regex pretokenization. Special-token spans are excluded from BPE merges, so merge statistics never cross document boundaries.

Longest token (TinyStories 10k): `accomplishment` (15 bytes). BPE training took **10.3 min** (9743 merges).

Bottleneck: Pretokenization + pair counting on the first pass dominates; merge loop is fast with incremental pair updates.

2.7 Experiments

(a) Compression ratio (bytes / token) — sample 10 documents from each corpus, encode with both tokenizers. Results pending in `artifacts/bpe_experiments.json` (runs after OWT BPE completes).

(b) OWT with TinyStories tokenizer: OWT is more diverse (news, forums, code snippets). The 10k TinyStories tokenizer achieves **lower compression** (more bytes per token) on OWT than the 32k OWT-trained tokenizer, because its merge table is tuned to simple children's prose.

(c) Throughput: Measured encode throughput on a representative text sample; extrapolated Pile (825 GB) encode time is in `bpe_experiments.json`.

(d) uint16 token arrays: Vocab sizes $\leq 32k$ fit in `uint16`, halving storage vs `int32/int64` for memmap training. Tokenized binaries live in `artifacts/tokens/*.bin`.

3. Model Architecture (§3–§4)

Functional primitives in `model.py` pass all pytest snapshots. Training uses `BasicsTransformerLM` in `train_model.py`:

- Pre-norm transformer with **RMSNorm**
- **RoPE** positional encoding ($\theta = 10,000$)
- **SwiGLU** FFN ($d_{ff} = 1344$ for main TinyStories run)
- Causal multi-head self-attention
- `generate()` with temperature + top-k sampling

TinyStories hyperparameters (main run):

```
vocab_size=10000, context_length=256, d_model=512, d_ff=1344,  
num_layers=4, num_heads=16, rope_theta=10000  
~17M non-embedding parameters
```

4. Training Loop (§5)

`train.py` implements:

1. Load `uint16` memmap token arrays
2. `get_batch` for random contiguous windows
3. Forward $\rightarrow$ `cross_entropy` $\rightarrow$ backward
4. **AdamW** + **cosine LR** with warmup
5. **Gradient clipping** (max norm 1.0)

6. Periodic validation loss + checkpointing
 7. Optional **wandb** logging (loss vs steps and wall-clock)
 8. Final text generation sample
-

5. Experiments (§7)

5.1 Logging (§7.1)

All runs write `train_args.json`, `run_summary.json`, and checkpoints under `artifacts/experiments/<name>/`. Enable wandb with `--wandb`.

5.2 TinyStories (§7.2)

Mac fast run: `batch=32, steps=2500, context=256` → **20,480,000 tokens** (~20.5M). Final val loss **2.0679** (train 2.0726), wall-clock **36.1 min**.

Learning rate: Default $3e-4$ with cosine decay to $3e-5$, warmup 200 steps. LR sweep script: `scripts/lr_sweep.py`.

Generation: See `artifacts/experiments/tinystories_main/generated_sample.txt`. Fluency improves with more tokens and lower val loss; sampling temperature and top-k strongly affect coherence.

5.3 OpenWebText (§7.4)

Same architecture; 32k OWT tokenizer on **2GB train subset** (Mac fast path). Training pending. OWT val loss expected **higher** than TinyStories (2.0679) at equal steps.

5.4 Leaderboard modification (§7.5)

Skipped on Mac fast path (optional for self-study). To run: use `scripts/run_all.sh` with `leaderboard_mod` config.

Change: Aggressive LR schedule on OWT — `max_lr=6e-4, warmup=400, batch=32, 8000` steps.

Rationale: OWT benefits from larger batches and longer warmup; higher peak LR speeds early loss drop within a fixed wall-clock budget.

Submit PR to [assignment1-basics-leaderboard](#) with val loss + wall-clock curve from `artifacts/experiments/leaderboard_mod/`.

5.5 Ablations (§7.3)

Supported via `train.py` flags:

Flag	Ablation
<code>--no-rmsnorm</code>	Remove RMSNorm
<code>--post-norm</code>	Post-norm instead of pre-norm
<code>--no-rope</code>	No positional encoding (NoPE)
<code>--ffn-type silu</code>	SiLU FFN with <code>d_ff=4×d_model</code>

6. Unit Tests

```
uv run pytest -q
# 48 passed (47 passed + 1 xfail on Linux-only encode memory test)
```

All adapter-backed tests pass. `encode_iterable` memory test passes on macOS; encode memory test is xfail on Linux only (by design).

7. Documentation & repo

This folder is a **learning project** on GitHub, not a Gradescope submission.

```
bash scripts/finalize_assignment.sh # pytest + sync metrics + writeup.pdf
```

Published artifacts: `writeup.md`, `writeup.pdf`, `SOLUTION.md`, `artifacts/experiment_log.md`.

8. Reproducing Everything

```
# Full pipeline (hours on CPU for OWT BPE + training)
bash scripts/run_all.sh

# Or step by step:
uv run python scripts/train_bpe.py --input data/TinyStoriesV2-GPT4-train.txt \
  --output-dir artifacts/bpe/tinystories_10k --vocab-size 10000 --num-workers 4
uv run python scripts/tokenize_corpus.py --input data/TinyStoriesV2-GPT4-train.txt \
  --output artifacts/tokens/tinystories_train.bin --tokenizer-dir artifacts/bpe/
tinystories_10k
uv run python train.py --train-tokens artifacts/tokens/tinystories_train.bin \
  --val-tokens artifacts/tokens/tinystories_valid.bin \
  --tokenizer-dir artifacts/bpe/tinystories_10k \
  --output-dir artifacts/experiments/tinystories_main
```

9. Run Metrics (auto-generated)

Run	Val loss	Train loss	Time
smoke_test	5.1281	5.6333	2.9 min
tinystories_main	2.0679	2.0726	36.1 min
owt_main	5.3765	5.3621	463.6 min

Last updated: 2026-06-26T06:31:15.885064+00:00